Annexe 17.7 - Patterns PHP / PDO / sécurité de référence

Ces extraits sont les **modèles à reproduire** dans l'application (lot L8->L12 du planning) ; ils ont été écrits en cohérence avec le DDL, les procédures et les triggers réellement testés (`sql/01...04`, 26/26 tests conformes). Ils ne sont pas encore exécutés ici : `php -l` et les tests PHPUnit sont à courir par l'équipe sur son environnement PHP 8.2.

1. Connexion PDO centralisée (`app/Config/Database.php`)

```
<?php
declare(strict_types=1);

namespace App\Config;

use PDO;

final class Database
{
    private static ?PDO $pdo = null;

    public static function pdo(): PDO
    {
        if (self::$pdo instanceof PDO) {
            return self::$pdo;
        }
        $cfg = require __DIR__ . '/env.php';        // hors dépôt (modèle : env.example.php)
        $dsn = sprintf('mysql:host=%s;port=%d;dbname=%s;charset=utf8mb4',
                       $cfg['db_host'], $cfg['db_port'], $cfg['db_name']);
        self::$pdo = new PDO($dsn, $cfg['db_user'], $cfg['db_pass'], [
            PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,   // jamais de warning muet
            PDO::ATTR_EMULATE_PREPARES   => false,                    // requêtes préparées
RÉELLES (SEC-01)
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
            PDO::ATTR_STRINGIFY_FETCHES  => false,
            PDO::MYSQL_ATTR_INIT_COMMAND => "SET NAMES utf8mb4 COLLATE utf8mb4_unicode_ci",
        ]);
        return self::$pdo;
    }
}
```

2. Repository : appel de procédure stockée avec paramètres liés

```
<?php
declare(strict_types=1);

namespace App\Repository;

use App\Model\SearchQuery;
use PDO;
```

```
final class ProduitRepository
{
    public function __construct(private readonly PDO $pdo) {}

    /** Catalogue/recherche/filtres : tout le SQL dynamique vit DANS la procédure. */
    public function search(SearchQuery $q, bool $admin = false): array
    {
        $stmt = $this->pdo->prepare(
            'CALL sp_search_products(?, ?, ?, ?, ?, ?, ?, ?, ?)'
        );
        $stmt->execute([
            $q->keyword, $q->categoryId, $q->priceMin, $q->priceMax,
            $q->inStockOnly ? 1 : 0, $q->sort, $q->page, $q->perPage, $admin ? 1 : 0,
        ]);
        $lignes = $stmt->fetchAll();
        $total  = 0;
        if ($stmt->nextRowset()) {
            $total = (int) $stmt->fetchColumn();
        }
        return ['lignes' => $lignes, 'total' => $total];
    }
}
```

3. Écriture multi-tables transactionnelle + traduction des rejets de règles

```
<?php
declare(strict_types=1);

namespace App\Repository;

use App\Model\Commande;
use App\Security\RegleInterdite;
use App\Security\StockInsuffisant;
use App\Security\AccesRefuse;
use PDO;
use PDOException;

final class CommandeRepository
{
    /** Codes renvoyés par la base -> objets métier (seule traduction autorisée du rejet). */
    private const TRADUCTIONS = [
        'STOCK_INSUFFISANT'            => StockInsuffisant::class,
        'ACCES_NON_AUTORISE'           => AccesRefuse::class,
        'RB04_COMMANDE_DOIT_CONTENIR_AU_MOINS_UNE_LIGNE' => RegleInterdite::class,
        'RB05_QUANTITE_DOIT_ETRE_SUPERIEURE_A_ZERO'       => RegleInterdite::class,
        'RB06_PRIX_ET_QUANTITE_NON_MODIFIABLE'            => RegleInterdite::class,
        'RB11_TRANSITION_STATUT_INTERDITE'                => RegleInterdite::class,
        'RB15_MONTANT_CALCULE_INTERDIT'                   => RegleInterdite::class,
        'PRODUIT_SUPPRIME_DU_CATALOGUE'                   => RegleInterdite::class,
        'RB19_PANIER_VIDE'                                => RegleInterdite::class,
    ];
```

```
    public function __construct(private readonly PDO $pdo) {}

    public function confirmerDepuisPanier(int $clientId, string $adresse, array $lignes, bool
$payee): Commande
    {
        $json = json_encode(
            array_map(static fn (array $l) => ['id_produit' => (int) $l['id_produit'],
                                               'quantite'  => (int) $l['quantite']], $lignes),
            JSON_THROW_ON_ERROR
        );

        $this->pdo->beginTransaction();
        try {
            $stmt = $this->pdo->prepare(
                'CALL sp_create_order_from_basket(?, ?, ?, ?, @id, @num, @montant, @code)'
            );
            $stmt->execute([$clientId, $adresse, $json, $payee ? 1 : 0]);

            $lecture = $this->pdo->query('SELECT @id AS id, @num AS numero,
                                                 @montant AS montant, @code AS code')->fetch();
            if ($lecture['code'] !== 'OK') {
                throw $this->traduire((string) $lecture['code']);
            }
            $this->pdo->commit();
            return Commande::depuisLigne($lecture);
        } catch (\Throwable $e) {
            $this->pdo->rollBack();          // aucune trace partielle : jamais de "commande sans
lignes"
            throw $e;
        }
    }

    private function traduire(string $code): \RuntimeException
    {
        $classe = self::TRADUCTIONS[$code] ?? RegleInterdite::class;
        return new $classe($code);           // le SQL et ses détails ne remontent JAMAIS à l'IHM
(SEC-12)
    }
}
```

> Note d'exécution : selon le pilote, le `OUT` des procédures peut être lu par session (`SELECT @id`) comme

> ci-dessus, ou passé en paramètres et relu dans le même appel ; l'important, pour le jury, est qu'il n'y ait

> **aucune** concaténation dans le texte SQL et **un seul** `ROLLBACK` possible.

4. Authentification : `password_hash` / `password_verify`

```
<?php
declare(strict_types=1);

namespace App\Security;
```

```
final class Hasher
{
    private const OPTIONS = ['cost' => 12];

    public function hacher(string $clair): string
    {
        $hash = password_hash($clair, PASSWORD_BCRYPT, self::OPTIONS);
        if ($hash === false) {                      // échec explicite, jamais un mot de passe en
 clair
            throw new \RuntimeException('Hachage impossible : algorithme indisponible.');
        }
        return $hash;
    }

    public function verifier(string $clair, string $hash): bool
    {
        return password_verify($clair, $hash);       // comparaison en temps constant
    }

    public function doitRehacher(string $hash): bool
    {
        return password_needs_rehash($hash, PASSWORD_BCRYPT, self::OPTIONS);
    }
}
```

```
// AuthController : le message est le même pour « email inconnu » et « mot de passe erroné »
(SEC-04)
$stmt = $this->clients->requireStatement();           // CALL sp_get_credentials(?)
$stmt->execute([$email]);
[$idClient, $hash, $statut] = array_values($stmt->fetch() ?: [null, null, 'INCONNU']);

if ($idClient === null || $statut === 'INCONNU' || $statut === 'COMPTE_BLOQUE'
    || !$this->hasher->verifier($motDePasse, (string) $hash)) {
    $this->temporiserApresEchec();                    // SEC-06
    $this->flash('danger', 'Email ou mot de passe incorrect.');
    return $this->redirect('/connexion', 303);
}
$this->auth->ouvreSessionClient((int) $idClient);     // session_regenerate_id(true) à l'intérieur
```

5. Sessions et contrôle d'accès (`app/Security/Auth.php`)

```
<?php
declare(strict_types=1);

namespace App\Security;

final class Auth
{
    public const VIE_INACTIF = 1800;      // 30 min
    public const VIE_MAX     = 43200;     // 12 h

    public function demarrer(): void
    {
```

```
        if (session_status() === PHP_SESSION_ACTIVE) {
            return;
        }
        session_name('MINISHOPSESSID');
        session_set_cookie_params([
            'lifetime' => 0, 'path' => '/', 'domain' => '',
            'secure'   => !APP_DEV,
            'httponly' => true,
            'samesite' => 'Lax',
        ]);
        session_start();
        $this->expireSiTropVieux();
    }

    public function ouvreSessionClient(int $id): void
    {
        session_regenerate_id(true);                              // anti-fixation (SEC-05)
        $_SESSION = ['user_id' => $id, 'role' => 'client',
                     'created' => time(), 'last' => time(), 'ip' => self::ip()];
    }

    public function exigeClient(): int
    {
        if (($_SESSION['role'] ?? null) !== 'client' || empty($_SESSION['user_id'])) {
            $_SESSION['after'] = $_SERVER['REQUEST_URI'] ?? '/';
            header('Location: /connexion', true, 303);
            exit;
        }
        $_SESSION['last'] = time();
        return (int) $_SESSION['user_id'];
    }

    public function exigeAdmin(string ...$roles): int
    {
        if (($_SESSION['role'] ?? null) !== 'admin'
            || ($roles && !in_array($_SESSION['admin_role'] ?? '', $roles, true))) {
            header('Location: /connexion?esp=admin', true, 303);
            exit;
        }
        return (int) $_SESSION['admin_id'];
    }

    public function fermer(): void
    {
        $_SESSION = [];
        if (ini_get('session.use_cookies')) {
            $p = session_get_cookie_params();
            setcookie(session_name(), '', time() - 42000, $p['path'], $p['domain'], $p['secure'],
$p['httponly']);
        }
        session_destroy();                                    // déconnexion réelle : le panier
part aussi
    }

    private function expireSiTropVieux(): void
    {
```

```
        $now = time();
        $create = (int) ($_SESSION['created'] ?? $now);
        $last  = (int) ($_SESSION['last'] ?? $now);
        if ($now - $last > self::VIE_INACTIF || $now - $create > self::VIE_MAX) {
            $this->fermer();
            session_start();
        }
    }

    public static function ip(): string
    {
        return (string) ($_SERVER['REMOTE_ADDR'] ?? '');
    }
}
```

6. CSRF et échappement

```
<?php
declare(strict_types=1);
// app/Security/csrf.php
function jeton_csrf(): string
{
    if (empty($_SESSION['_csrf'])) {
        $_SESSION['_csrf'] = bin2hex(random_bytes(32));
    }
    return $_SESSION['_csrf'];
}

function verifie_jeton_csrf(?string $recu, bool $consommer = false): void
{
    if (!is_string($recu) || !hash_equals($_SESSION['_csrf'] ?? '', $recu)) {
        Journal::ecrit('CSRF_REJECTED', ['route' => $_SERVER['REQUEST_URI'] ?? '']);
        http_response_code(419);
        exit('Session expirée ou requête non autorisée. Rechargez la page.');
    }
    if ($consommer) {
        unset($_SESSION['_csrf']);      // jeton à usage unique pour la validation de commande
    }
}
```

```
<?php
declare(strict_types=1);
// app/Security/xss.php - l'unique sortie HTML autorisée dans les vues (SEC-03)
function e(?string $valeur): string
{
    return htmlspecialchars((string) $valeur,
                            ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML5, 'UTF-8');
}
```

7. JavaScript : mise à jour du panier **sans** HTML concaténé

```
// public/js/panier.js - module ES2022 ; la sécurité est côté serveur, ici on ne fait que
l'affichage.
const zone = document.querySelector('#recap-panier');

async function changerQuantite(idProduit, quantite) {
  const champ = document.querySelector(`[data-qte="${idProduit}"]`);
  const reponse = await fetch('/panier/quantite', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json', 'X-CSRF-Token': jeton() },
    body: JSON.stringify({ id_produit: Number(idProduit), quantite: Number(quantite) })
  });
  const data = await reponse.json();

  if (!reponse.ok) {                        // STOCK_INSUFFISANT, PRODUIT_INTROUVABLE, ...
    afficherErreur(zone, data.erreur, data.reste);   //textContent : aucun HTML injecté -> XSS
neutralisé
    champ.max = String(data.stock ?? 0);
    return;
  }
  rendreRecapitulatif(zone, data);          // idem : construction par nœuds / textContent
  document.querySelector('#badge-panier').textContent = String(data.nb_articles);
}

document.addEventListener('input', (e) => {
  const champ = e.target.closest('[data-qte]');
  if (!champ) return;
  const max = Number(champ.max || 999);
  const valeur = Math.max(1, Math.min(max, Number(champ.value || 1)));
  if (valeur !== Number(champ.value)) champ.value = String(valeur);   // confort, pas sécurité
  debounced(champ);
});
```

8. Vue d'erreur / code HTTP (aucune fuite technique)

```
// app/View/erreur.php
http_response_code($statut);            // 400, 403 -> 404 pour ressource d'autrui, 409, 419, 500,
 503
echo '<h1>' . e($titre) . '</h1>';
echo '<p>' . e($message) . '</p>';      // message métier traduit ; jamais le SQL, jamais un
chemin serveur
if (APP_DEV) {
    echo '<pre>' . e($trace) . '</pre>';   // trace réservée au développement local
}
```

9. Ce que la revue de code doit refuser (check-list du relecteur)

Interdit	Pourquoi	Détecté par
`$sql = "... WHERE id = $id"`	injection SQL	`grep` CI n° 1 + revue
`mysql_*`, `mysqli_query($conn, "...$var...")`	hors `CT-02` (PDO imposé)	revue

`echo $produit['nom']` sans `e()`	XSS	`grep` CI n° 2
`$_POST['montant']` utilisé pour un prix	violerait `RB-15`/`RB-06`	revue
`md5()`/`sha1()` pour un mot de passe	`RB-12`	`grep` + test
requête `UPDATE produit SET stock = ?` depuis un contrôleur	la base décide (`RB-03`/`RB-18`) : passer par `sp_adjust_stock`	revue
affichage de `PDOException::getMessage()` à l'utilisateur	fuite d'information (`SEC-12`)	revue
`id` de commande pris dans l'URL sans filtre `id_client`	IDOR (`SEC-08`)	`grep` CI n° 5